T.LAKSHMI PRASAD NAIDU 192324187

Ques7on 1: Online Symposium Portal

The given registration form contains basic input controls and uses POST submission, but it can be improved by using semantic HTML5 elements, accessible labels, validation, and a clear document structure.

1. Iden7fy appropriate HTML5 seman7c elements missing in the structure.

The original form contains only the <form>, <input>, and <button> elements. Several HTML5 semantic elements can be added to give the page a meaningful structure.

- <header> can contain the symposium title, logo, and introductory information.
- <main> can contain the primary registration content.
- <section> can group the symposium registration form as one logical section.
- <footer> can contain contact information, copyright information, or other supporting details.
- <label> should be associated with every form control so that users, including screen-reader users, know what information is required.
- <fieldset> and <legend> can group related form controls and provide a clear description of the group.

These elements improve structure, accessibility, maintainability, and the semantic meaning of the webpage.

2. Modify the structure to make it standards-compliant.

A standards-compliant version can use a complete HTML5 document structure, a language declaration, character encoding, a viewport declaration, semantic elements, labels, and appropriate input types.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Symposium Registration</title>
</head>
<body>
    <header>
        <h1>College Online Symposium</h1>
        <p>Register to participate in the symposium.</p>
    </header>

  <main>
    <section aria-labelledby="registration-heading">
      <h2 id="registration-heading">Registration Form</h2>

      <form action="/register" method="post">
        <fieldset>
          <legend>Student Details</legend>

          <p>
            <label for="name">Full Name:</label>
            <input type="text" id="name" name="name"
```

```
                    autocomplete="name" required>
</p>

            <p>
              <label for="email">Email Address:</label>
<input type="email" id="email" name="email"
                    autocomplete="email" required>
</p>

            <button type="submit">Submit Registration</button>
</fieldset>
        </form>
      </section>
</main>

  <footer>
    <p>&copy; College Symposium</p>
  </footer>
</body>
</html>
```

The DOCTYPE identifies HTML5, while lang, charset, viewport, labels, required attributes, and semantic elements make the structure clearer and more accessible.

3. Interpret the HTTP request generated during form submission.

When the student clicks the submit button, the browser collects the successful form controls and sends an HTTP POST request to the URL specified by action, which is /register.

A simplified request is:

POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=StudentName&email=student%40example.com The

important parts are:

- POST: the HTTP method used to submit the form data.
- /register: the server endpoint receiving the data.
- Host: identifies the server being contacted.
- Content-Type: describes the format of the submitted form data.
- Request body: contains the name and email values.

Unlike GET, POST normally places the submitted form data in the request body rather than in the URL query string. The server can then validate the data, process the registration, store it if appropriate, and generate a response.

4. Analyze the HTTP response cycle from server to browser.

The response cycle begins after the browser sends the POST request.

1. The server receives the request at the /register endpoint.
2. The server reads and validates the submitted name and email.

3. The server performs the required application processing, such as creating a registration record.
4. The server generates an HTTP response containing a status code, headers, and possibly an HTML response body.
5. The browser receives the response and interprets it.
6. Depending on the response, the browser may display a confirmation page, show validation errors, or follow a redirect.

For example, a successful response might use:

HTTP/1.1 200 OK
Content-Type: text/html or the server

may use a redirect such as:

HTTP/1.1 303 See Other
Location: /registration-success

The browser then requests the redirected resource and displays the resulting page. Thus, HTTP communication follows a request-response cycle between the browser and server.

5. Suggest improvements for usability and accessibility.

The registration portal can be improved in the following ways:

• Provide visible labels for all form controls.
• Use required where a field must be completed.
• Use type="email" so the browser can perform basic email validation.
• Use meaningful autocomplete values such as name and email.
• Provide clear instructions and examples where necessary.
• Keep error messages specific and close to the relevant field.
• Ensure sufficient keyboard accessibility and a logical tab order.
• Use semantic HTML elements such as header, main, section, and footer.
• Ensure adequate text contrast and readable font sizes.
• Do not rely only on colour to communicate errors or required information.
• Use responsive design so the form works on mobile and desktop screens.
• Preserve user-entered values when validation fails so the user does not need to re-enter them.

These changes make the portal easier to understand and operate for a wider range of users.

Ques7on 2: Cross-Browser Compa7bility Issue

The given page has incomplete HTML elements and improper nesting. Different browsers may attempt error recovery differently, so correcting the document structure is important for consistent rendering.

1. Iden7fy improper nes7ng and missing closing tags.

The snippet contains an <h1> element that is opened but not explicitly closed before the element begins. The paragraph also does not have a closing tag. The structure is therefore incomplete.

The problematic portion is:

  <h1>Welcome
  About us

A corrected version should explicitly close the heading and paragraph:

<h1>Welcome</h1>
About us

Explicitly closing elements and maintaining proper nesting makes the document structure unambiguous and easier for browsers, validators, assistive technologies, and developers to interpret.

2. Analyze how different browsers may interpret this structure.

HTML parsers are designed to recover from certain authoring errors. When tags are omitted or incorrectly nested, a browser may automatically insert or close elements while constructing its internal document tree.

Because error recovery is involved, malformed markup can produce differences in the resulting DOM, layout, styling, or interaction across browsers and browser versions. CSS rules may also behave unexpectedly if the DOM tree is not the one the developer intended.

Therefore, relying on browser error recovery is not a reliable way to create a cross-browser website. Valid, wellstructured HTML reduces ambiguity and makes the rendered result more consistent.

3. Iden7fy missing seman7c elements (e.g., header, sec7on).

The page uses a generic even though the content has meaningful structural roles.

Suitable semantic elements include:

• <header> for the introductory area or page heading.
• <main> for the primary content of the page.
• <section> for a distinct thematic group such as the company introduction.
• <footer> for supporting information such as contact or copyright details.

Using semantic elements communicates the purpose of each part of the document and can improve accessibility and maintainability.

4. Provide a corrected W3C-compliant HTML structure.

A corrected HTML5 structure can be written as:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Company</title>
</head>
<body>
    <header>
        <h1>Welcome</h1>
    </header>

  <main>
    <section aria-labelledby="about-heading">
      <h2 id="about-heading">About Us</h2>
      <p>About us</p>
    </section>
</main>

  <footer>
```

```
    <p>&copy; Company</p>
  </footer>
</body>
</html>
```

This version has a valid HTML5 document declaration, explicit closing tags, correct nesting, a language attribute, metadata, and meaningful semantic elements.

5. Suggest techniques to ensure cross-browser compa7bility.

The following practices help improve cross-browser compatibility:

- Use valid, standards-compliant HTML and CSS.
- Validate HTML and CSS during development.
- Use semantic HTML rather than browser-specific markup.
- Test the website in multiple browsers and viewport sizes.
- Use responsive CSS and appropriate media queries.
- Avoid obsolete or browser-specific features unless there is a justified compatibility strategy.
- Use feature detection when JavaScript functionality depends on browser capabilities.
- Keep browsers and development/testing tools up to date.
- Use progressive enhancement so the basic content remains usable even when advanced features are unavailable.
- Test keyboard navigation and accessibility across browsers.
- Check the final DOM and computed styles when diagnosing layout differences.

These practices reduce differences caused by malformed markup, unsupported features, or browser-specific behaviour.

Ques7on 3: Form Submission Failure

The form is intended to submit login information, but the button is explicitly configured as a non-submit button. The method also has important implications for how the data is transmitted.

1. Iden7fy issues in the form configura7on.

The main issue is the button:

```
<button type="button">Login</button>
```

A button with type="button" does not submit its containing form by default. Therefore, clicking it does not trigger normal form submission.

Other considerations include:

- The form uses method="GET", which places submitted form data in the URL query string.
- For login credentials, GET is inappropriate because the username and password can become part of the URL and may be exposed through browser history, logs, bookmarks, or other intermediaries.
- The form does not show labels for the username and password fields.
- The fields do not explicitly use required validation.
- The example does not indicate use of HTTPS, which is essential for protecting credentials in transit.

2. Analyze why the HTTP request is not triggered.

The immediate reason is the button type. The form contains:

```
<button type="button">Login</button>
```

The value type="button" creates a general-purpose button rather than a form submission button. Clicking it therefore does not invoke the browser's normal form submission process.

If the button were changed to:

<button type="submit">Login</button>

the browser would submit the form to the URL in the action attribute, subject to normal form validation. Alternatively, JavaScript could call form.submit() or requestSubmit(), but using a normal submit button is the simplest approach when no custom behaviour is required.

3. Interpret the role of GET method in this scenario.

GET is designed primarily for requesting a resource and can also submit form data by appending it to the URL as a query string.

For example, the form could generate a request resembling:

```
GET /submit?username=student&password=example HTTP/1.1
```

This is unsuitable for a login form because sensitive credentials should not be placed in the URL. URLs can be stored in browser history, server logs, proxy logs, analytics systems, bookmarks, or copied and shared accidentally.

Therefore, GET may be appropriate for non-sensitive searches or retrieval-oriented forms, but it should not be used to transmit passwords.

4. Propose a corrected implementa7on.

A basic corrected implementation is:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
</head>
<body>
    <main>
        <section aria-labelledby="login-heading">
            <h1 id="login-heading">Login</h1>

      <form action="/submit" method="post">
        <p>
          <label for="username">Username:</label>
          <input type="text" id="username" name="username"
autocomplete="username" required>          </p>

        <p>
          <label for="password">Password:</label>
          <input type="password" id="password" name="password"
autocomplete="current-password" required>          </p>

        <button type="submit">Login</button>
      </form>
    </section>
```

```
  </main>
</body>
</html>
```

The important changes are method="post", type="submit", associated labels, required validation, and appropriate autocomplete values.

5. Suggest improvements for secure data transmission.

For secure login data transmission, the following measures should be used:

- Use HTTPS so the connection is encrypted in transit.
- Submit credentials using POST rather than placing them in the URL with GET.
- Validate input on the server as well as in the browser.
- Store passwords using a strong password-hashing algorithm on the server, never as plain text.
- Use secure session management after successful authentication.
- Set appropriate cookie security attributes such as Secure and HttpOnly where applicable.
- Protect authentication endpoints against common attacks such as credential stuffing and excessive automated login attempts.
- Avoid placing passwords or other secrets in URLs, page titles, client-side logs, or error messages.
- Use appropriate security headers and keep the server-side framework and dependencies updated.

The key principle is that the browser form is only one part of the security system. Transport security, server-side validation, authentication, password storage, and session management all need to work together.

Reference to the Assessment Brief

This document answers all three application-based questions given in Assessment Tool 1 for Internet Programming (CSA43), covering HTML5 structure, standards compliance, HTTP request/response behaviour, cross-browser compatibility, form submission, and secure data transmission.